МОДУЛЬ 3

Обработка текста как задача искусственного интеллекта

Разработчик интеллектуальных решений
Базовый уровень для студентов СПО

Обработка текста как задача искусственного интеллекта

01 Практикум: текст как данные, частоты слов, признаки текста, ручная разметка и готовый шаблон классификации.

02 Работа строится как небольшой эксперимент: сначала формулируется правило разметки, затем создаётся базовая модель, после чего её решения проверяются на новых сообщениях.

Планируемые результаты

01 объяснять, почему компьютер не «понимает» текст напрямую и зачем нужны числовые признаки.

02 выделять класс объекта и признаки текста.

03 считать частоты слов и интерпретировать модель «мешок слов».

04 запускать готовый конвейер CountVectorizer → MultinomialNB.

Что понадобится

01 компьютер с браузером и средой Google Colab или Jupyter Notebook.

02 Python 3 и библиотека scikit-learn в учебной среде.

03 файл не требуется: набор данных встроен в шаблон занятия.

04 рабочий лист или этот документ для записи выводов.

2. Вступление-обращение

01 Сегодня вы увидите, как короткие фразы превращаются в числа и становятся входом для модели машинного обучения.

02 Особое внимание уделите пограничным сообщениям.

3.1. Текст как данные

01 Для человека фраза несёт смысл.

02 В этом практикуме объектом является одно обращение пользователя.

03 Разделение на токены зависит от выбранного способа обработки.

04 Представьте набор карточек.

3.2. Частоты слов и «мешок слов»

```
from collections import Counter
import re
texts = [
"оплата не проходит",
"оплата картой не проходит",
"оплата отклонена",
]
words = re.findall(r"[а-яё]+", " ".join(texts).lower())
print(Counter(words).most_common(3))
```

Один из простых способов представить текст — посчитать, сколько раз каждое слово встречается в документе.

Проверьте понимание теории

01 Что будет выведено кодом из примера 1?

02 А) [(«оплата», 3), («не», 2), («проходит», 2)]

03 Почему слова «отправили» и «отправлен» могут стать разными признаками?

04 А) В простом векторизаторе это разные последовательности символов

3.3. Готовый конвейер

```
model = Pipeline([
("vectorizer", CountVectorizer()),
("classifier", MultinomialNB()),
])
model.fit(train_texts, train_labels)
predictions = model.predict(test_texts)
```

В практикуме используется Pipeline : первый шаг CountVectorizer строит словарь и матрицу частот, второй шаг MultinomialNB обучается связывать эти частоты с классами.

3.4. Как оценивать качество

01 Accuracy — доля верных предсказаний среди всех проверочных примеров.

02 Матрица ошибок показывает, какие истинные классы модель перепутала и с какими классами.

03 Precision отвечает на вопрос: насколько точны прогнозы выбранного класса.

04 Recall отвечает на вопрос: какую долю объектов выбранного класса модель смогла найти.

4. Практический блок (1/2)

```
from collections import Counter
import re
sample_texts = [
"курьер опаздывает",
"курьер не позвонил",
"доставка задерживается",
"оплата не проходит",
"оплата картой отклонена",
]
def tokenize(text):
return re.findall(r"[а-яё]+", text.lower())
```

Фрагмент примера. Полный код приведён в тексте лекции.

4. Практический блок (2/2)

01 Примеры: Назовите два слова, которые помогают распознать класс «доставка», Почему слово «не» само по себе плохо различает классы?, Предложите одно улучшение токенизации или подготовки текста.

02 При ответе на третий вопрос сравните сообщения «курьер не позвонил» и «оплата не проходит»: один и тот же токен встречается в разных тематических классах.

03 _____

Обязательное задание на анализ кода (1/4)

```
vectorizer = CountVectorizer()
X_train = vectorizer.transform(train_texts)
print(X_train.shape)
```

Что произойдёт при выполнении фрагмента?

A) Будет выведен список классов

Обязательное задание на анализ кода (2/4)

```
train_texts = [
"не могу войти в личный кабинет",
"забыл пароль от аккаунта",
"код подтверждения не приходит",
"учетная запись заблокирована",
"как изменить пароль",
"ошибка авторизации при входе",
"не удается зарегистрироваться",
"почта не принимается при входе",
"восстановите доступ к профилю",
"после обновления меня выбросило из аккаунта",
"логин не найден системой",
"двухфакторный код не работает",
```

Фрагмент примера. Полный код приведён в тексте лекции.

Обязательное задание на анализ кода (3/4) · таблица 1/3

Показатель	Ваш результат
Размер обучающего набора	
Размер тестового набора	

Обязательное задание на анализ кода (3/4) · таблица 2/3

Показатель	Ваш результат
Accuracy	
Количество ошибок	

Обязательное задание на анализ кода (3/4) · таблица 3/3

Показатель	Ваш результат
Класс с минимальным recall	

Обязательное задание на анализ кода (4/4)

Текст	Истинный класс	Прогноз	Вероятная причина

Обязательное задание на исправление ошибок · таблица 1/2

Сравнение	Базовая модель	Улучшенная модель
Количество обучающих текстов		
Accuracy		

Обязательное задание на исправление ошибок · таблица 2/2

Сравнение	Базовая модель	Улучшенная модель
Количество ошибок		
Изменение качества	—	

5. Дополнительные задачи

```
my_texts = [
# добавьте 6 собственных сообщений
]
print(improved_model.predict(my_texts))
```

Эти задачи не влияют на обязательный результат занятия и предназначены для повышенной нагрузки.

Во всех дополнительных экспериментах сохраняйте тот же тестовый набор.

Дополнительная задача 2. Униграммы и биграммы

01 Замените CountVectorizer() на CountVectorizer(ngram_range=(1, 2)) .

02 При параметре ngram_range=(1, 2) признаками становятся и отдельные токены, и пары соседних токенов.

03 Дополнительная задача 3.

04 Выберите одну ошибку модели и опишите её возможные последствия для пользователя и службы поддержки.

6. Критерии оценивания · таблица 1/3

Задача	Проверяемый результат	Баллы
№1	Размечены 9 сообщений, указаны слова-подсказки, объяснена неоднозначность	0–2
№2	Запущен подсчёт частот, корректно интерпретированы признаки и ограничение слова «не»	0–3

6. Критерии оценивания · таблица 2/3

Задача	Проверяемый результат	Баллы
№3	Шаблон запущен, зафиксированы метрики и размеры наборов	0–4
№4	Найдены ошибки модели, объяснены причины, исправлен код	0–5

6. Критерии оценивания · таблица 3/3

Задача	Проверяемый результат	Баллы
№5	Модель улучшена, результаты сравнены, вывод содержит ограничение эксперимента	0–6
Итого	Максимум	20

7. Чек-лист результата

01 ☐ Я могу объяснить, что такое документ, токен, признак и класс.

02 ☐ Я вручную разметил примеры и заметил неоднозначный текст.

03 ☐ Я запустил подсчёт частот слов.

04 ☐ Я выполнил готовый шаблон CountVectorizer → MultinomialNB .

8. Итог урока

01 Сегодня вы сделали полный мини-цикл классификации текста

02 Разметили данные → превратили слова в частоты → обучили модель → проверили качество → разобрали ошибки → улучшили набор данных.

03 Качество каждого этапа влияет на следующий: непоследовательная разметка создаёт противоречивые примеры, ограниченные признаки теряют часть информации, а маленький тестовый набор даёт нестабильную оценку.

04 Кратко сформулируйте один вывод о данных и один вывод о модели.

9. Самопроверка

01 Что делает CountVectorizer ?

02 _____

_____

03 Для чего нужна ручная разметка?

04 _____

_____

Краткий глоссарий

01 Документ. Один отдельный текстовый объект: сообщение, отзыв, письмо.

02 Токен. Элемент текста, обычно слово или часть слова.

03 Признак. Числовая характеристика, которую получает модель.

Практическая работа

01 Задача №1

02 Задача №2

03 Задача №3

04 Задача №4

Проверка результата

01 Сопоставление результата с условием

02 Проверка на исходных и изменённых данных

03 Объяснение решения своими словами